COMPILER DESIGN LAB | BCSE307P

Single-Pass Code Generation for If-Else statements

Using syntax-directed translation and backpatching

Aarushi Jha | 24BCB0025
Mitali Agrawal | 24BCB0034
Shashank Pandurang Kamat | 24BCB0062
Faculty: Dr. Bhawana Tyagi

25% **Review 1**
Lexer and parser

75% **Review 2**
TAC for expressions and conditions

100% **Review 3**
If-else translation, tests, demo

OVERVIEW

Project roadmap

25%

Review 1

- Problem and objectives
- Literature survey
- Grammar for if-else
- Lexer and parser

75%

Review 2

- Three-address code (TAC)
- Attributes and backpatch helpers
- Arithmetic expressions
- Short-circuit ||, && and !
- Novelty

100%

Review 3

- If and if-else translation
- Nested ifs, dangling else
- Running the TAC, errors
- Tests and demo
- Future scope: optimization

Scope: assignments, arithmetic, boolean conditions (||, &&, !) and if / if-else, translated in one pass.

Review 1

Problem, literature survey, lexer and parser

25% of the implementation

In this part

1. The problem
2. Objectives
3. Literature survey
4. Grammar
5. Pipeline
6. Lexer
7. Parser
8. Dangling else
9. Output

The problem we are solving

Many compilers build a full syntax tree first, then walk it again to fix jump targets.

1. **Heavy memory**
   The whole program tree stays in memory.

2. **Extra passes**
   Forward jumps need another walk over the tree.

3. **Short-circuit rules**
   In A || B, skip B when A is already true.

Example

```
if (a < b || c == 10) {
    x = a + b * 2;
}
```

What should happen

If a < b is true, the program must not even look at c == 10. The compiler has to turn this rule into jumps, and it has to do it while reading the code once.

Project objectives

1 Clear grammar

Write an unambiguous grammar for arithmetic, assignments, conditions and nested if-else.

2 Lexical analyzer

Turn raw text into tokens, track line and column numbers, skip spaces and comments.

3 Recursive descent parser

Check syntax with hand-written functions. Precedence comes from the call order.

4 Ready for code generation

Prepare single-pass TAC with backpatching for forward jumps in if-else.

Literature survey: compiler phases



Cuts source text into tokens such as if, x, 10 and +.

Checks that the tokens follow the grammar rules.

Attaches code-emitting actions to grammar rules.

Simple instructions with at most three operands.

Leaves jump targets blank and fills them in later.

Built in Review 1

Built in Review 2 and 3

Two ways to translate code

	Traditional multi-pass (AST)	Single-pass backpatching (ours)
Passes	Many: build the tree, then walk it	One: code is emitted while parsing
Memory	Higher: stores every tree node	Lower: a flat list of instructions
Forward jumps	Fixed in later walks or label tables	Filled in on the fly by backpatch()
Short-circuit	Needs extra tree pruning	Falls out of truelist and falselist

The language grammar

Context-free grammar

```
Stmt       -> ID = Expr ;
            | if ( Bool ) Stmt [ else Stmt ]
            | { StmtList }
Bool       -> BoolTerm ( || BoolTerm )*
BoolTerm   -> BoolFactor ( && BoolFactor )*
BoolFactor -> ! BoolFactor | ( Bool )
            | Expr [ RELOP Expr ]
Expr       -> Term ( ( + | - ) Term )*
Term       -> Factor ( ( * | / ) Factor )*
Factor     -> ID | NUM | - Factor | ( Expr )
```

Precedence

Lower rules bind tighter: * beats +, && beats ||.

No left recursion

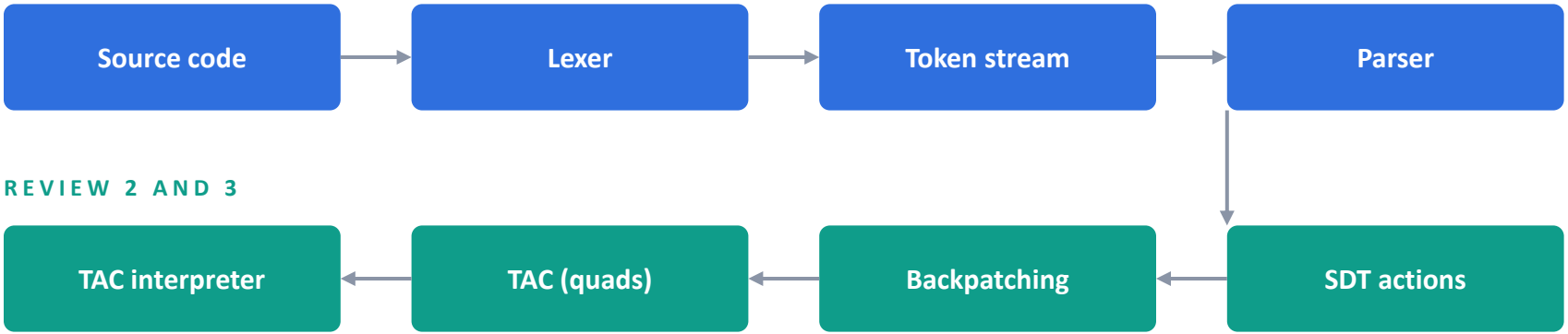
Safe for top-down parsing.

One else rule

else pairs with the nearest if.

Pipeline: how the parts connect

REVIEW 1



Lexer: text into tokens

- Reads the source one character at a time
- Skips spaces, new lines and // comments
- Groups characters into tokens
- Reports illegal or malformed input

`x = a + 2;`

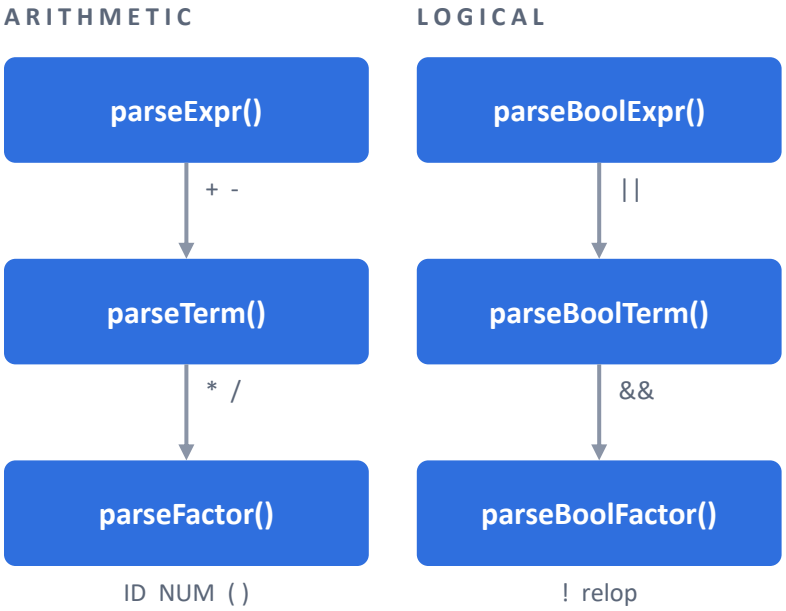
ID	ASSIGN	ID	PLUS	NUM	SEMI
x	=	a	+	2	;

full_implementation.cpp: Token, Lexer::scanTokens (condensed)

```
struct Token {
    TokenType type;  // IF, ID, NUM, RELOP ...
    string text;     // characters matched
    int line, col;   // used in error messages
};

if (isAlphaCh(ch) || ch == '_') {
    // read letters, digits and _ into word
    TokenType type = TokenType::ID;
    if (word == "if")        type = TokenType::IF;
    else if (word == "else") type = TokenType::ELSE;
    tokens.push_back({type, word, line, col});
}
```

Parser: grammar as functions



Deeper functions run first, so they bind tighter.

Shape of every level (simplified)

```
ExprAttr parseExpr() {
    ExprAttr e1 = parseTerm();
    while (next is + or -) {
        ExprAttr e2 = parseTerm();
        ...combine e1 and e2...
    }
    return e1;
}
```

Statements follow the same idea: parseStatement() looks at one token and calls parseAssignment(), parseIf() or parseBlock().

Dangling else, solved

```
if (a > b)
    if (c > d)
        x = 1;
    else
        x = 2;
```

The else belongs to the nearest if

How the parser does it

- parseIf() reads the condition and one statement.
- If that statement is itself an if, the inner call runs first.
- The inner call checks for else before returning.
- So the else is taken by the closest if.

full_implementation.cpp: parseIf

```
StmtAttr s1 = parseStatement();
if (peek().type == TokenType::ELSE) {
    ...parse the else part...
}
```

Review 1 output

Input program

```
if (a < b || c == 10) {
    x = a + b * 2;
} else {
    x = 0;
}
```

The parser accepts valid code. Screenshot taken from implementation_phase1.cpp.

